

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Práctica 2:

Quetzal Space Defender - Side Scroller

PONDERACIÓN: 5 pts

 **Tiempo estimado: 20 hrs**

1. MARCO FORMATIVO.....	
1.1. Valor.....	
1.2. Competencia(s).....	
1.3. Objetivo SMART.....	
2. Enunciado de la Práctica.....	
2.1 Descripción del problema a resolver.....	
2.2 Alcance de la práctica.....	
3. Entregables.....	
4. Material de apoyo.....	
5. Recursos y herramientas a utilizar.....	
6. Cronograma.....	
7. Rúbrica de Calificación.....	
7.1 Requisitos para optar a la calificación.....	
7.2 Resumen de Puntuaciones.....	
Detalle de la Calificación.....	

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Disciplina	El estudiante sigue el cronograma de forma disciplinada para poder cumplir exitosamente con la entrega

1.2. Competencia(s)

Tipo de Competencia	
Competencia General	Comunica soluciones técnicas complejas a audiencias diversas utilizando documentación técnica, presentaciones y demostraciones interactivas adaptando el nivel de detalle según el contexto y audiencia objetivo
Competencia Específica	Evalúa arquitecturas de software y calidad de código empleando métricas de software, patrones de diseño y técnicas de testing para garantizar mantenibilidad, escalabilidad y robustez del sistema

1.3. Objetivo SMART

SMART	Definición	Objetivo redactado
Específico (¿Qué?)	El objetivo es concreto y tangible.	Los estudiantes serán capaces de crear una aplicación con interfaz gráfica, en la cual se implementen conceptos básicos de POO e hilos.
Medible (¿Cuánto?)	El objetivo tiene una medida objetiva de éxito.	Alcanzar un 80% de éxito en la realización de la práctica y el proyecto 2 del curso.
Alcanzable (¿Cómo?)	El objetivo debe ser posible con los recursos disponibles.	Utilizando únicamente las herramientas de laboratorio disponibles en clase.

Realista (¿Para qué?)	El objetivo contribuye a metas más amplias.	Con habilidades prácticas que fortalezcan la formación en informática y sistemas, preparándose para cursos más avanzados en desarrollo de software.
A Tiempo (¿Cuándo?)	El objetivo tiene fecha límite o mejor aún un cronograma de hitos de progreso	Cumplir este aprendizaje en dos semanas, las asignadas para esta actividad.

2. Enunciado de la Práctica

2.1 Descripción del problema a resolver

La empresa **Quetzal** requiere un simulador de combate espacial donde el jugador atraviesa un cinturón de asteroides y naves enemigas. A diferencia de las versiones anteriores, el movimiento será **horizontal** (de izquierda a derecha), exigiendo una gestión precisa de colisiones y generación de objetos en el borde derecho de la pantalla.

2.2 Personalización del piloto (dificultades)

Menú Principal: Botones para Jugar, Crear Piloto, Top de Puntajes y Salir.

Al registrar o elegir un piloto, el usuario debe seleccionar un modelo de nave que define la dificultad del juego. Cada modelo ajusta los tiempos de `sleep()` en los hilos de movimiento y disparo:

Dificultad	Tipo de Nave	Características	Lógica de Hilos
Fácil	Explorador	Alta velocidad / Disparo lento	Movimiento fluido, pero recarga de proyectil cada 2 seg.
Normal	Caza Estelar	Velocidad media / Disparo medio	Equilibrio entre movimiento y cadencia (1 seg).
Difícil	Acorazado	Velocidad baja / Disparo rápido	Nave pesada (difícil esquivar), pero ráfagas cada 0.3 seg.

Mecánicas del Campo de Batalla

- **Movimiento Horizontal:** Los enemigos y asteroides aparecen en el eje X máximo y se desplazan hacia el X mínimo.
- **Hilos:** Cada proyectil disparado por el jugador y cada enemigo en pantalla debe ser un hilo independiente que actualice su posición (x, y).
- **Premios y Obstáculos:**
 - **Snitch Espacial:** Otorga 150 puntos y destruye a todos los enemigos visibles.
 - **Asteroide (Bludger):** Ralentiza la nave del jugador o la bloquea por 2 segundos.
 - **Contenedor Quaffle:** Suma 10 puntos al puntaje actual.

2.3 Requerimientos técnicos

- **Gráficas:** Uso de **JFreeChart** para mostrar el desempeño de los mejores puntajes.
- **Exportación de Reportes:** Se debe utilizar exclusivamente las librerías derivadas de java.io.* para la generación de archivos.
- **Nota:** Para el PDF, se recomienda escribir un archivo **HTML/CSS** mediante FileWriter o PrintWriter que contenga los datos y la gráfica (exportada como imagen con ImageIO), el cual pueda ser visualizado y guardado como PDF desde el navegador.
- **Persistencia:** Uso de vectores para almacenar el historial de partidas y pilotos.

Entregables

Describe los productos concretos que se espera que los estudiantes entreguen al finalizar la práctica. Pueden ser prototipos, informes técnicos, documentación, etc.

Ejemplo Software – Introducción a la Programación 1:

Tipo	Descripción
Repositorio en GitHub	Repositorio con el nombre IPC1B_NombreApellido_Carnet_1S26 que contenga el código fuente de la práctica en una carpeta <i>Practica2</i> .
Código Fuente	Implementación completa en Java , organizada y comentada siguiendo las buenas prácticas de programación.
Manual Técnico	Documento en Markdown con observaciones técnicas, tales como la estructura del proyecto, librerías implementadas, lógica general

	y métodos importantes utilizados.
Manual de Usuario	Documento en Markdown con instrucciones claras para interactuar con el sistema, incluyendo ejemplos de uso y capturas de pantalla.
Diagrama de Flujo	Representación gráfica del funcionamiento del sistema y los procesos principales.

3. Material de apoyo

- Deitel, P., & Deitel, H. (2016). *Cómo programar en Java (10ª edición)*. Pearson Education.
- Joyanes, L. *Java 2 – Manual de Programación*. McGrawHill.
- Documentación oficial de Java SE
- Tutoriales de W3Schools – Java

4. Recursos y herramientas a utilizar

- **Software:** Java JDK, NetBeans o IntelliJ IDEA.
- **Plataformas:** GitHub (para repositorio), UEDi (para entrega).
- **Lecturas recomendadas:** manuales de Java, artículos sobre POO e hilos.

5. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de la Práctica	14/03/26	14/03/26
Elaboración	14/03/26	27/03/26
Calificación	28/03/26	28/03/26

6. Rúbrica de Calificación

7.1 Requisitos para optar a la calificación

Antes de la evaluación de la práctica, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Software – Introducción a la Programación 1

Tema	Descripción	Cumple (Si/No)
Cumplimiento de la tecnología establecida	El desarrollo debe haberse realizado en Java y con uso de estructuras básicas (vectores, ciclos).	
Uso de herramientas requeridas	El proyecto debe compilar y ejecutarse en NetBeans, IntelliJ o similar.	
Gestión y entregas de práctica	El repositorio en GitHub debe contener el código fuente y commits periódicos que demuestren el desarrollo continuo.	
Documentación obligatoria	Se debe incluir Manual Técnico, Manual de Usuario y Diagrama de Flujo en Markdown.	
Pruebas y funcionalidad mínima	El sistema debe permitir la creación de personajes, la simulación de partidas, la generación del top de puntajes y la consulta del historial.	

7.2 Resumen de Puntuaciones

Software – Introducción a la Programación 1

Área	Puntos Totales	Puntos Obtenidos
1. Habilidades		
Uso de ciclos y estructuras (Vectores)	8	

Calidad de código (modularidad, DRY, legibilidad)	8	
Entrega & Documentación (repo ordenado, docs completas)	12	
Interactividad del menú (no funcional)	1	
Preguntas teóricas	20	
Sub-Total Habilidades	40	
2. Conocimiento		
Gestión de Personajes (con validaciones)	20	
Simulación de carreras	20	
Interacción y navegación	10	
Reportes (historial y top)	10	
Sub-Total Conocimiento	60	
TOTAL	100	

Detalle de la Calificación

	Criterio/Nivel			Nivel de evaluación		
No.	Criterio de evaluación	Punteo maximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Habilidades	40				
1.1	Uso de ciclos y estructuras (Vectores)	8	Descripción	El programa utiliza de forma correcta ciclos y estructuras de datos (vectores) para resolver los problemas planteados, optimizando la lógica y garantizando resultados coherentes.	El uso de ciclos y estructuras es inadecuado, genera errores de ejecución o no resuelve los problemas correctamente, evidenciando un manejo insuficiente de vectores.	
			Rango del punteo	8 - 5/	4 - 0/	
	Sub-Total de Puntos					
2	Conocimientos	60				
2.1	Creación de Personajes (con validaciones)	30	Descripción	El sistema permite crear y leer los datos de los personajes, aplicando validaciones que impiden datos duplicados o inconsistentes. Todas las operaciones se ejecutan sin errores y mantienen la	El sistema presenta fallos en las operaciones de creación y lectura y carece de validaciones básicas o permite el ingreso de datos inválidos, generando	

				integridad de la información.	inconsistencias en la información de los personajes.	
			Rango del punteo	30 - 18/	17 - 0/	
	Sub-Total de Puntos					
	Total					